

Lecture 3

Clustering and Probability

An introduction to unsupervised learning
and a review of core concepts in probability

Keji He @ SDU-AI
keji.he@sdu.edu.cn

Roadmap

- K-means Clustering
 - Scikit-Learn
 - Lloyd's Algorithm
 - Pixel K-Means
- Probability
 - Joint Distributions
- Wake Word Example

Questions

K-means Clustering

- K-means Clustering
 - Scikit-Learn
 - Lloyd's Algorithm
 - Pixel K-Means
 - Probability
 - Joint Distributions
 - Wake Word Example
- Questions

Prof. Gonzalez's Messy Biking Records

Professor Gonzalez has too many bikes



He has been recording the **speed** and **length** of his bike rides but not which bike he used.

He would like to answer questions like:

1. What is the **average ride time** for each bike?
2. Are there any rides that are **abnormally short or long**?
3. What is the **most likely bike** for each ride?

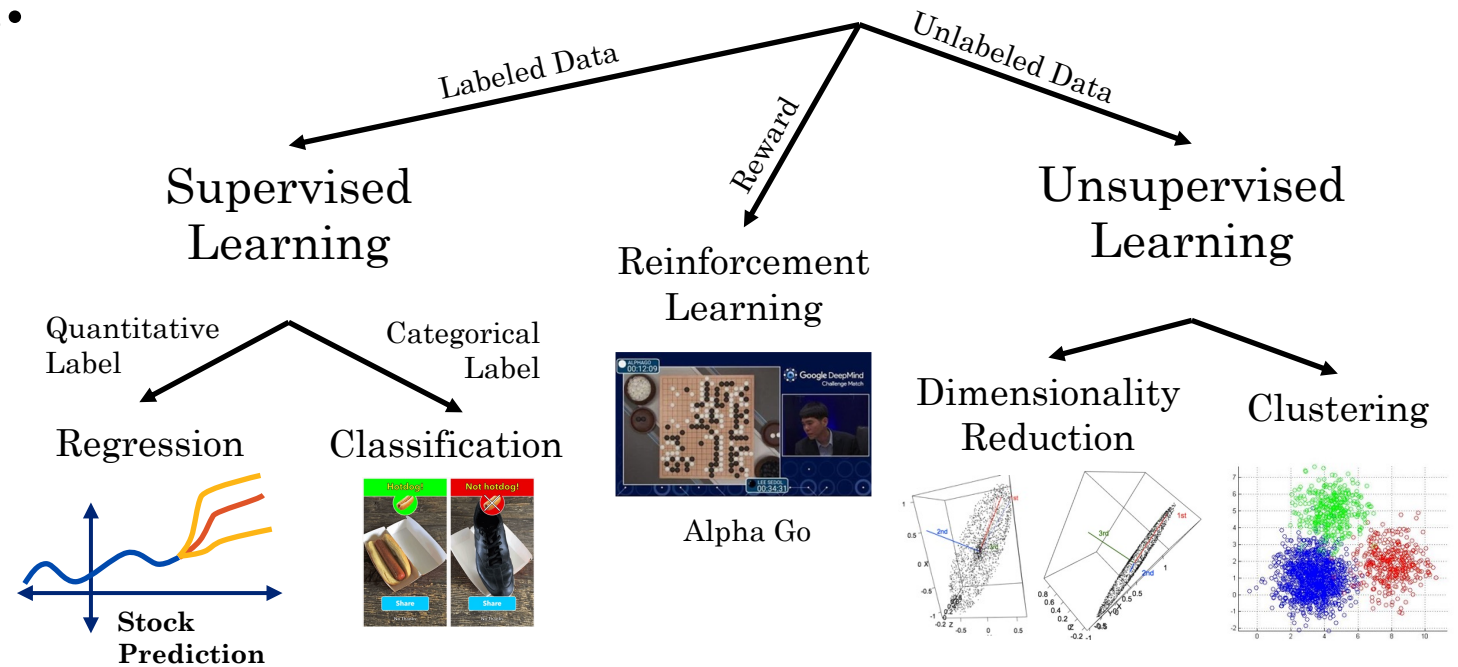


Learning Problem

We have **unlabeled data** and we would like to divide the records into 4 **groups (clusters)** corresponding to the four bikes.

- **Unsupervised Learning:**
we don't have the labels.
- **Clustering:**
we are trying to infer the **unobserved (*latent*)** bike choice.

Let's try
k-means clustering

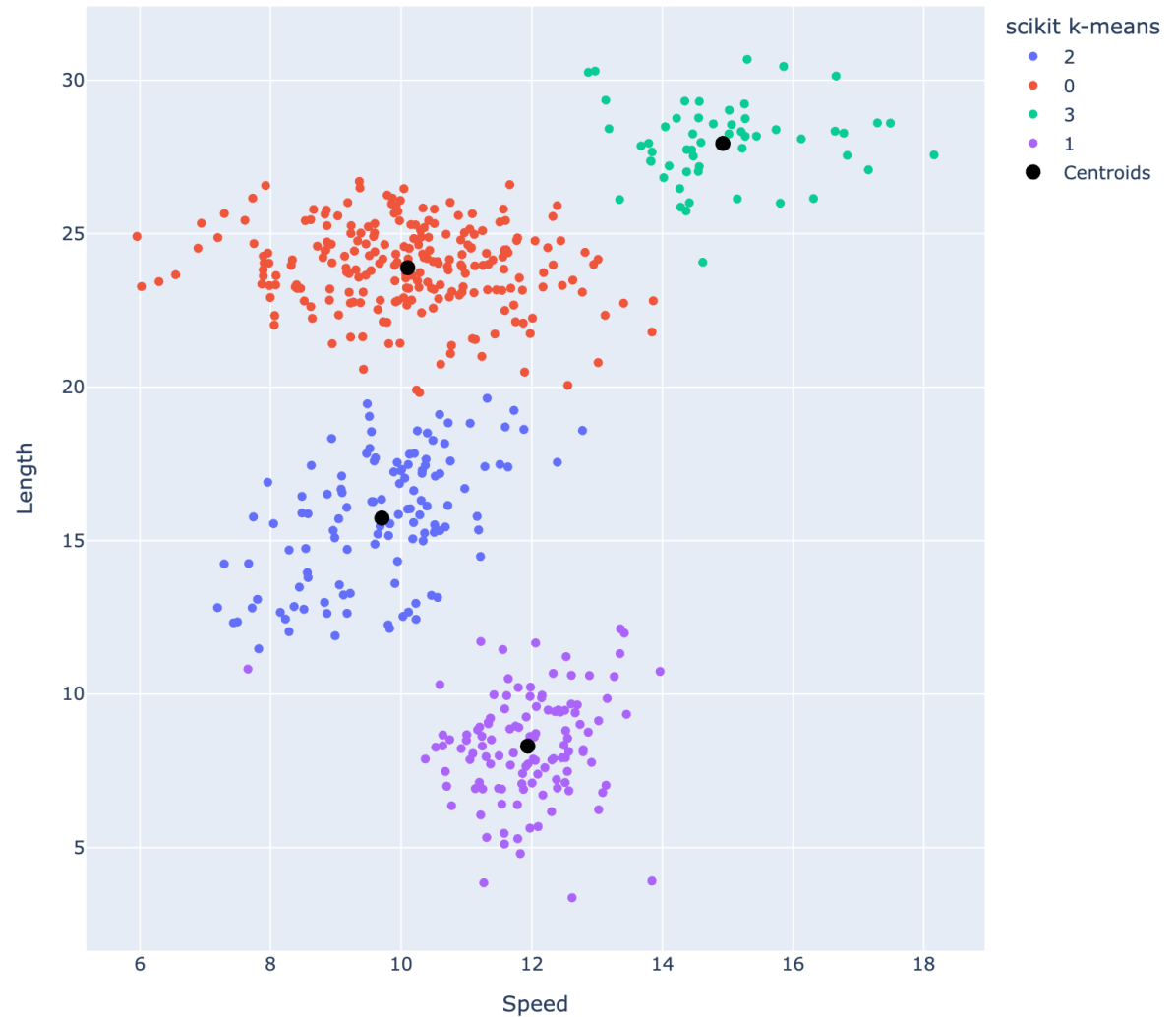


Demo

Using sklearn k-means clustering

SkLearn K-means Clustering

K-Means Clustering of Bike Segments



Clustering With K-Means in §

In the demo we wrote a few lines of code and ok
clustering of the ride-times.

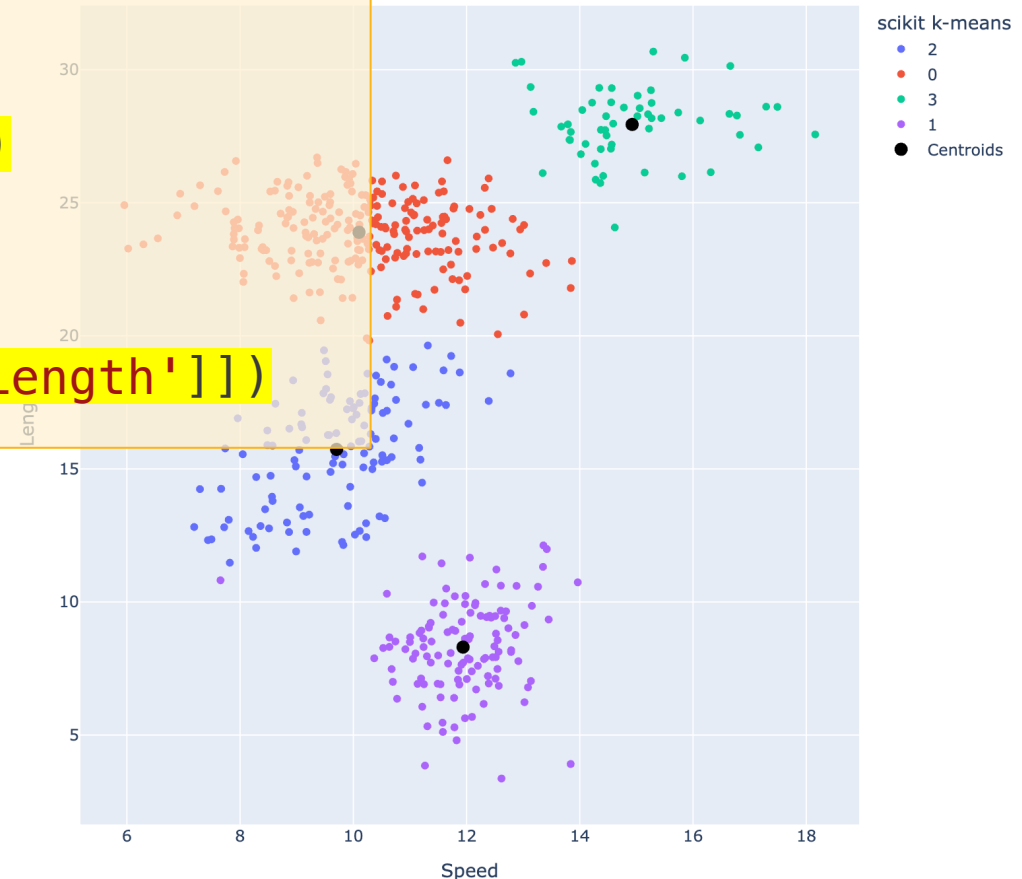
```
from sklearn.cluster import KMeans
# Create a KMeans model with 4 clusters
kmeans = KMeans(n_clusters=4, random_state=42)
# Fit the model to the data
kmeans.fit(bikes[['Speed', 'Length']])
# Predict the cluster assignments
bikes['c'] = kmeans.predict(bikes[['Speed', 'Length']])
```

Today we will learn how this algorithm works.

We will review concepts in probability and explore more general density estimation techniques.

速度 (Speed)	长度 (Length)	c (聚类标签)
12.5	10.2	0
25.8	45.3	2
11.9	9.8	0
18.2	22.1	1
15.5	30.4	3

K-Means Cluste



Lloyd's Algorithm

- Clustering
 - Scikit-Learn
 - K-means (Lloyd's) Algorithm
 - Pixel K-Means
 - Probability
 - Joint Distributions
 - Wake Word Example
- Questions

K-Means Clustering

Input: A collection of data points $\mathcal{D} = \{x_1, \dots, x_N\}$ where $x_n \in \mathbb{R}^D$.

Output: K cluster centers $\mu_k \in \mathbb{R}^D$ and an **assignment** $z_n \in \{1, \dots, K\}$ of each data point to one of the cluster centers.

- The means μ_k are **model parameters** of the model (z_n can be computed from μ) that are **fit to the data**.
- K is a **hyperparameter** which we need to select.

Objective: Each data point should be close to its assigned center.

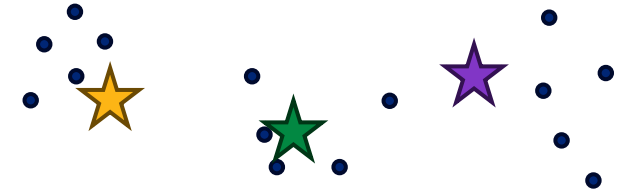
$$\arg \min_{\mu, z} \sum_{n=1}^N \|x_n - \mu_{z_n}\|_2^2$$

$(L_2\text{-Norm})^2$
Euclidean Distance Squared

$$\|a\|_2^2 = \sum_{d=1}^D a_d^2$$

K-Means Cluster (Lloyd's) Algorithm

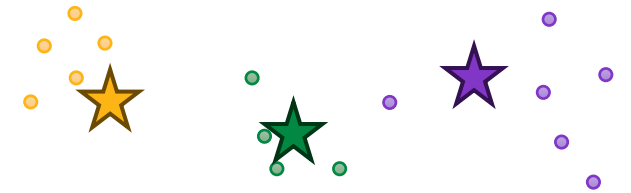
Initialization: Choose K points at random to be the initial cluster centers μ .



Iterate until convergence:

1. **Update Assignments:** Assign each point to its nearest cluster center.

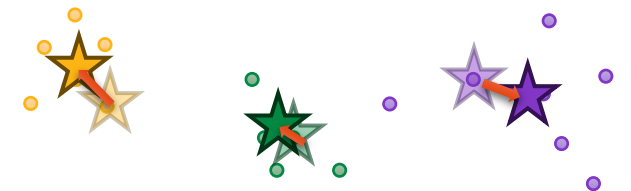
$$z = \arg \min_z \sum_{n=1}^N \|x_n - \mu_{z_n}\|_2^2$$



2. **Update Centers:** Recompute centers by averaging assigned points.

Why is this the mean of each cluster?

$$\mu = \arg \min_{\mu} \sum_{n=1}^N \|x_n - \mu_{z_n}\|_2^2$$



Updating the Cluster Centers

What is the value that minimizes?

$$\arg \min_{\mu} \sum_{n=1}^N \|x_n - \mu_{z_n}\|_2^2$$

We can re-arrange the objective to optimize with respect to μ :

$$\sum_{n=1}^N \|x_n - \mu_{z_n}\|_2^2 = \sum_{n=1}^N \sum_{d=1}^D (x_{nd} - \mu_{z_nd})^2 = \sum_{k=1}^K \sum_{i=1}^{N_k} \sum_{d=1}^D (x_{k_id} - \mu_{kd})^2$$

k_i is the i^{th} point in cluster k

Sum over **points in cluster k**

Sum over **clusters**

Minimizing the Transformed Objective

We take the derivative with respect to μ_{kd} :

$$\frac{\partial}{\partial \mu_{kd}} \sum_{k=1}^K \sum_{i=1}^{N_k} \sum_{d=1}^D (x_{k_id} - \mu_{kd})^2 = \sum_{i=1}^{N_k} -2(x_{k_id} - \mu_{kd})$$

Setting the derivative equal to zero and solving for μ_{kd} :

$$\sum_{i=1}^{N_k} -2(x_{k_id} - \mu_{kd}) = -2 \sum_{i=1}^{N_k} x_{k_id} + 2 N_k \mu_{kd} = 0$$

Cluster Mean
(Average Point)

$$\mu_{kd} = \frac{1}{N_k} \sum_{i=1}^{N_k} x_{k_id}$$



Is Lloyd's algorithm guaranteed to converge? Does it always produce an optimal clustering?

Convergence of K-Means

Is the k-means (Lloyd's) algorithm guaranteed to converge?

- Yes ☺! Why?

Alternating minimization always decreases the objective

$$z = \arg \min_z \sum_{n=1}^N \|x_n - \mu_{z_n}\|_2^2 \quad \text{and} \quad \mu = \arg \min_{\mu} \sum_{n=1}^N \|x_n - \mu_{z_n}\|_2^2$$

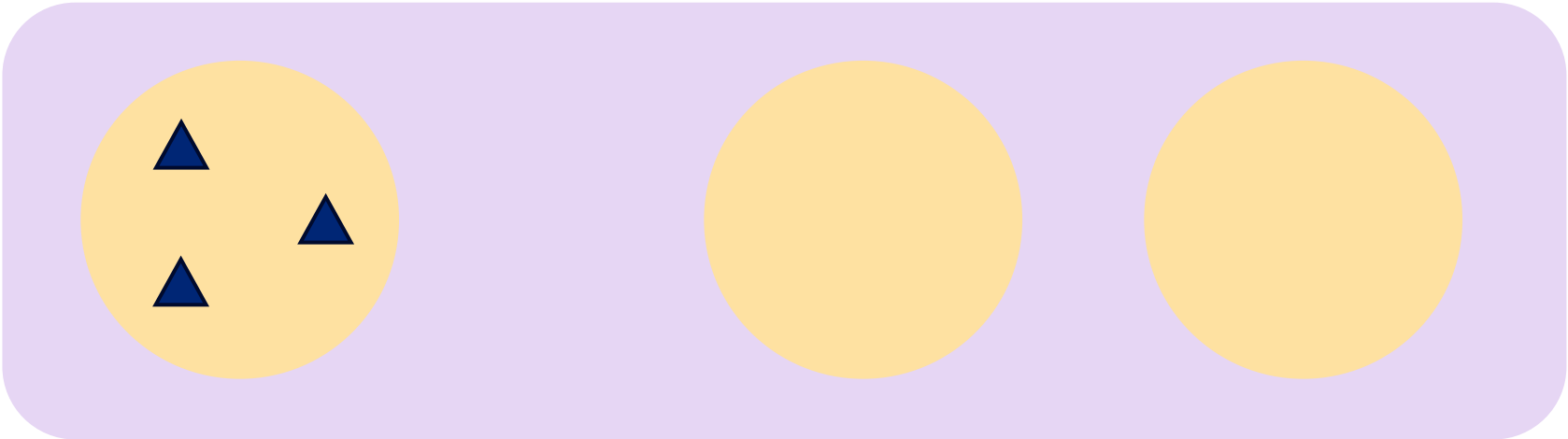
How do we know when it has converged?

- The **cluster assignments** (z) stop changing.

Do the final z^* and μ^* minimize the objective?

- No ☹! Can be **local minima**.

Initialization method 1
for cluster centers



Initialization method 2
for cluster centers

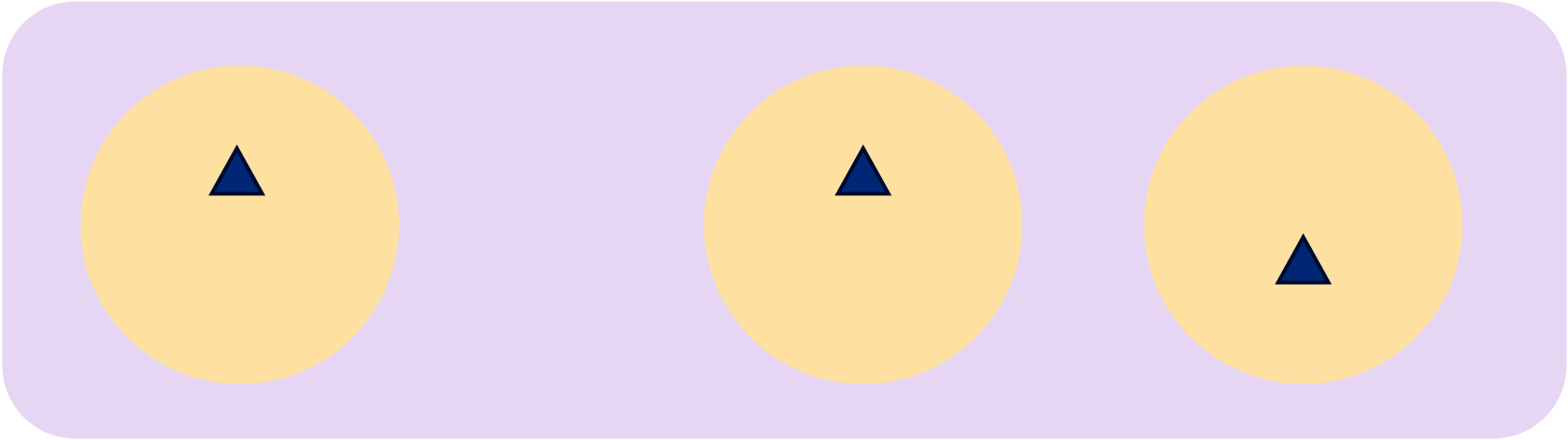
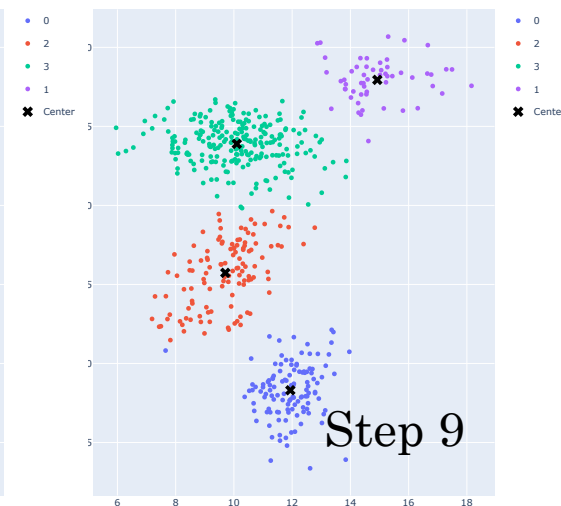
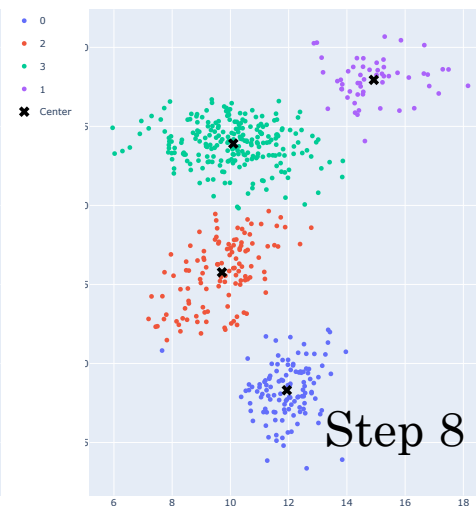
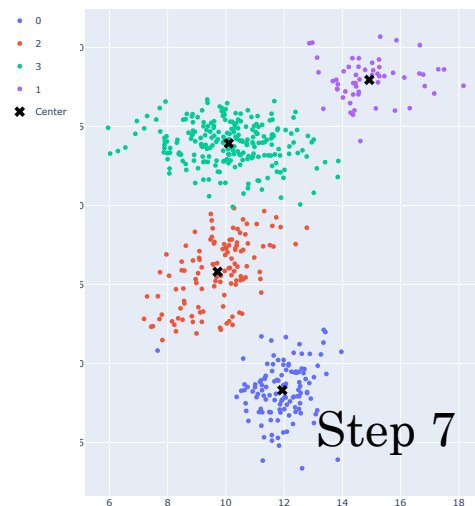
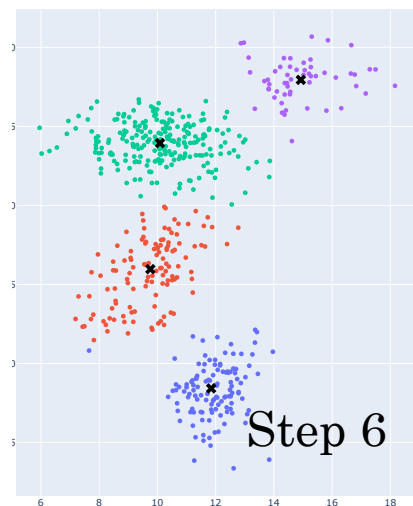
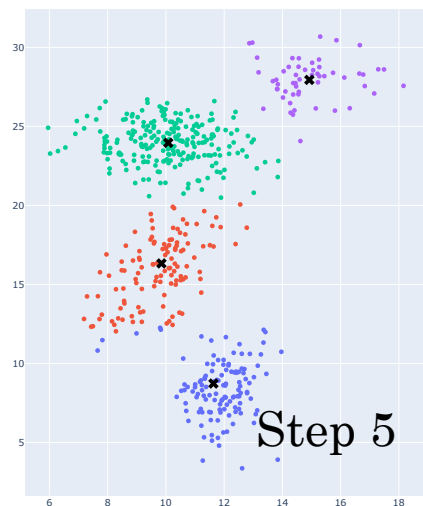
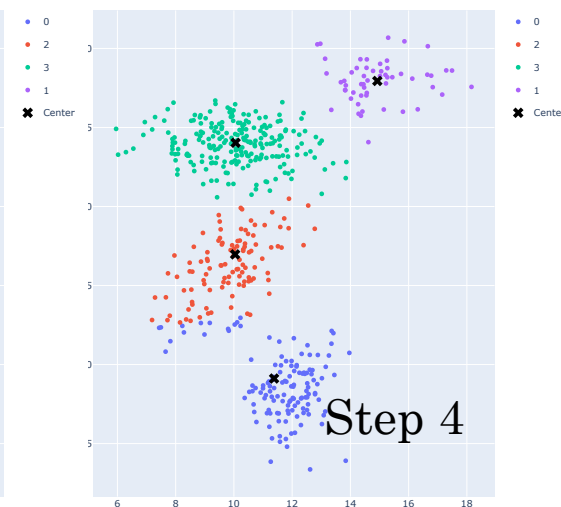
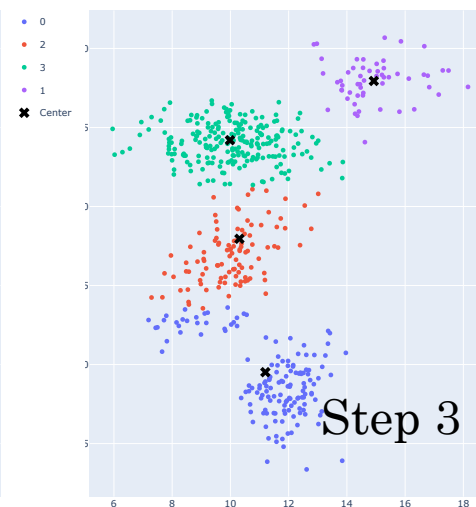
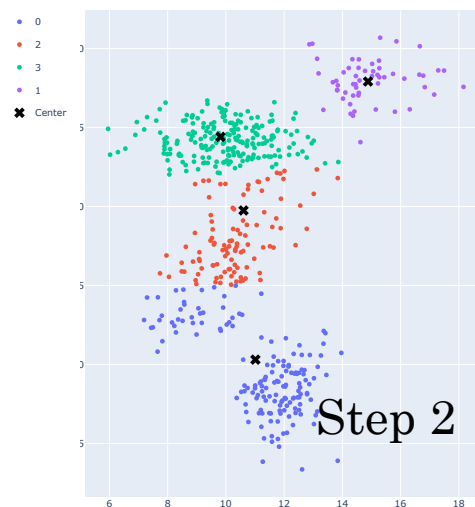
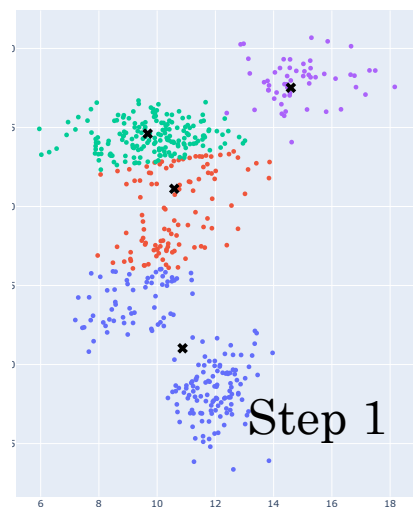
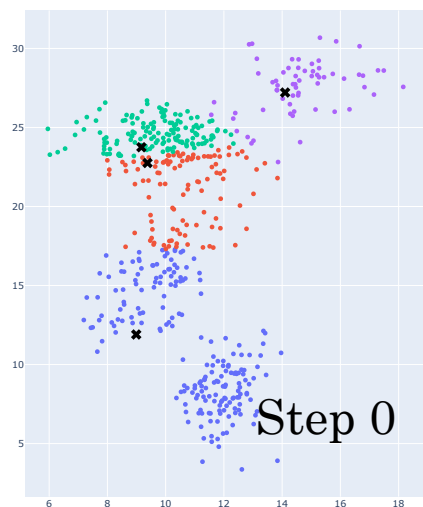


Illustration of Steps



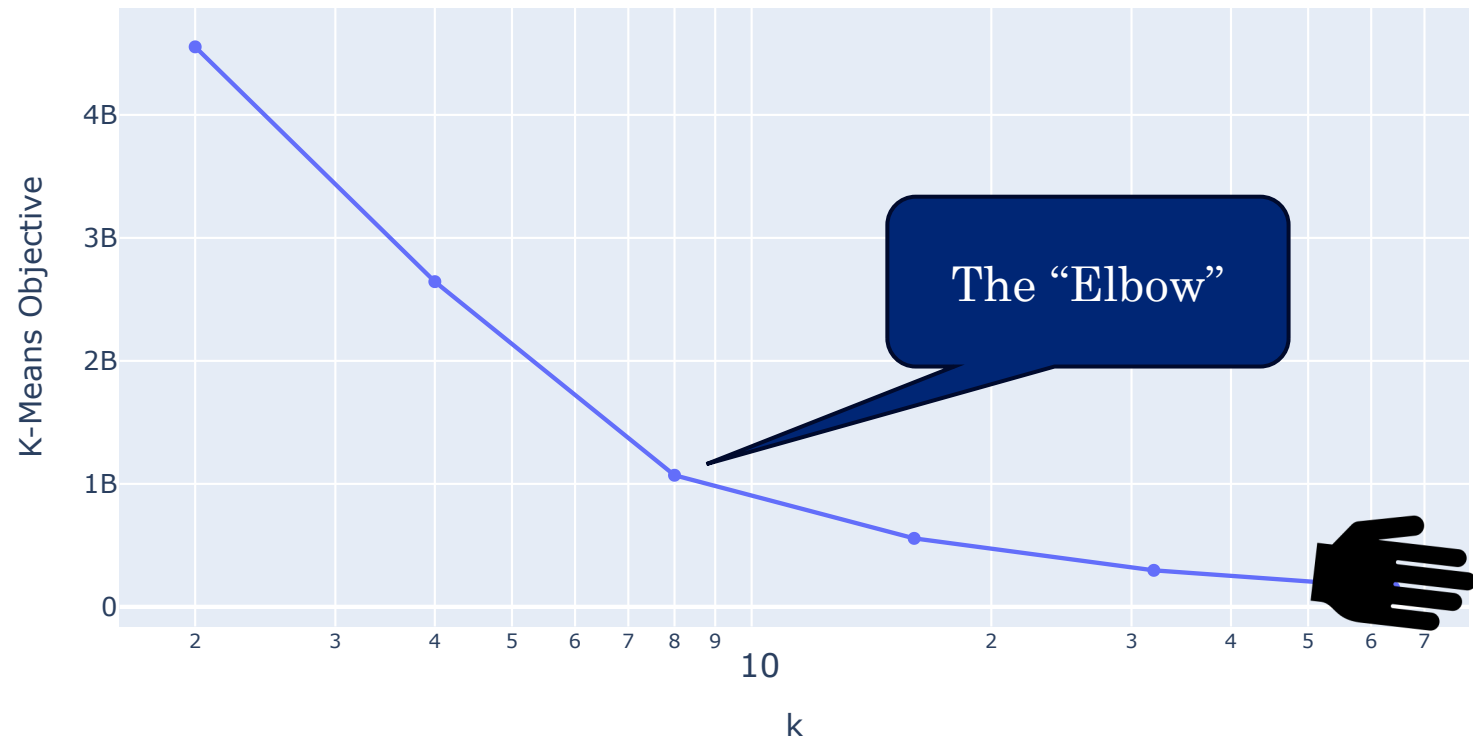
Choosing the Number of Clusters

There are several ways to select the **hyperparameter** K .

- Use **domain understanding** (e.g., 4 bikes)
- Use the “**elbow method**” to select the value of K with diminishing improvements in the objective:

$$\sum_{n=1}^N \|x_n - \mu_{z_n}\|_2^2$$

K-Means Objective vs Number of Clusters



Interpreting the Clusters

We used k-means to compute a **cluster assignment** for each ride.

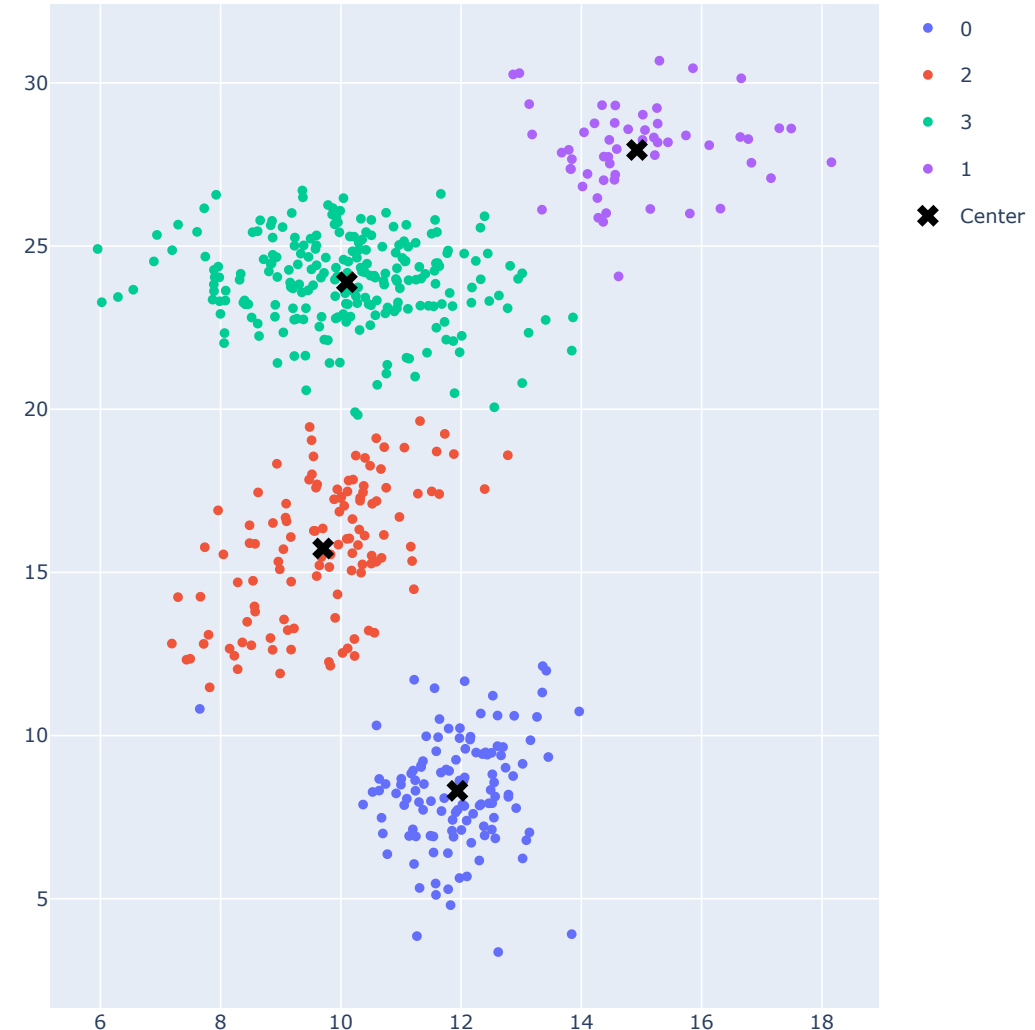
Which bike is each cluster?

- We don't know. Could ask for a few labels.

Does each cluster **represent a bike**?

- Maybe? But it could also correspond to **other factors** (e.g., group ride, traffic).

Use caution when interpreting clusters!



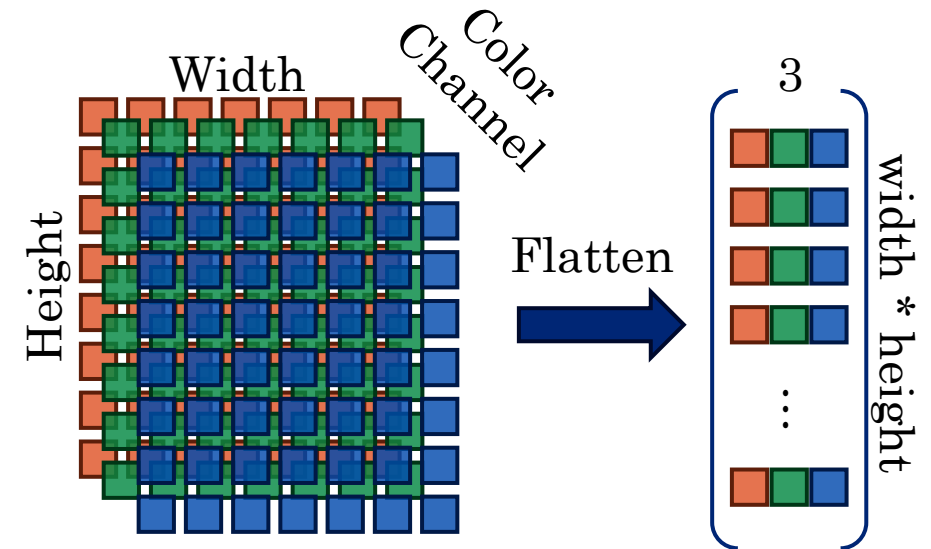
Pixel K-Means

- K-means Clustering
 - Scikit-Learn
 - Lloyd's Algorithm
 - Pixel K-Means
 - Probability
 - Joint Distributions
 - Wake Word Example
- Questions

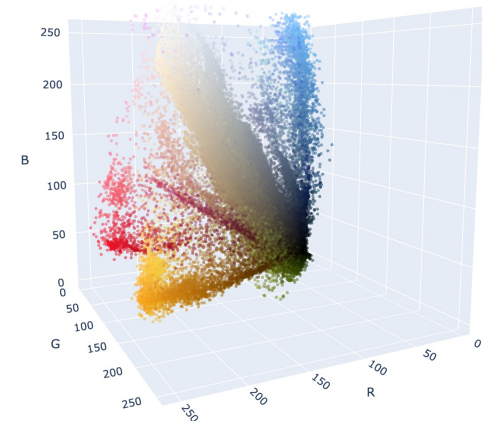
Pixel K-Means

The pixels in an image can be treated of as **vectors** in an **RGB vector space**.

- We can use k-means to compute the **clusters of colors** and render an image using just a few colors.



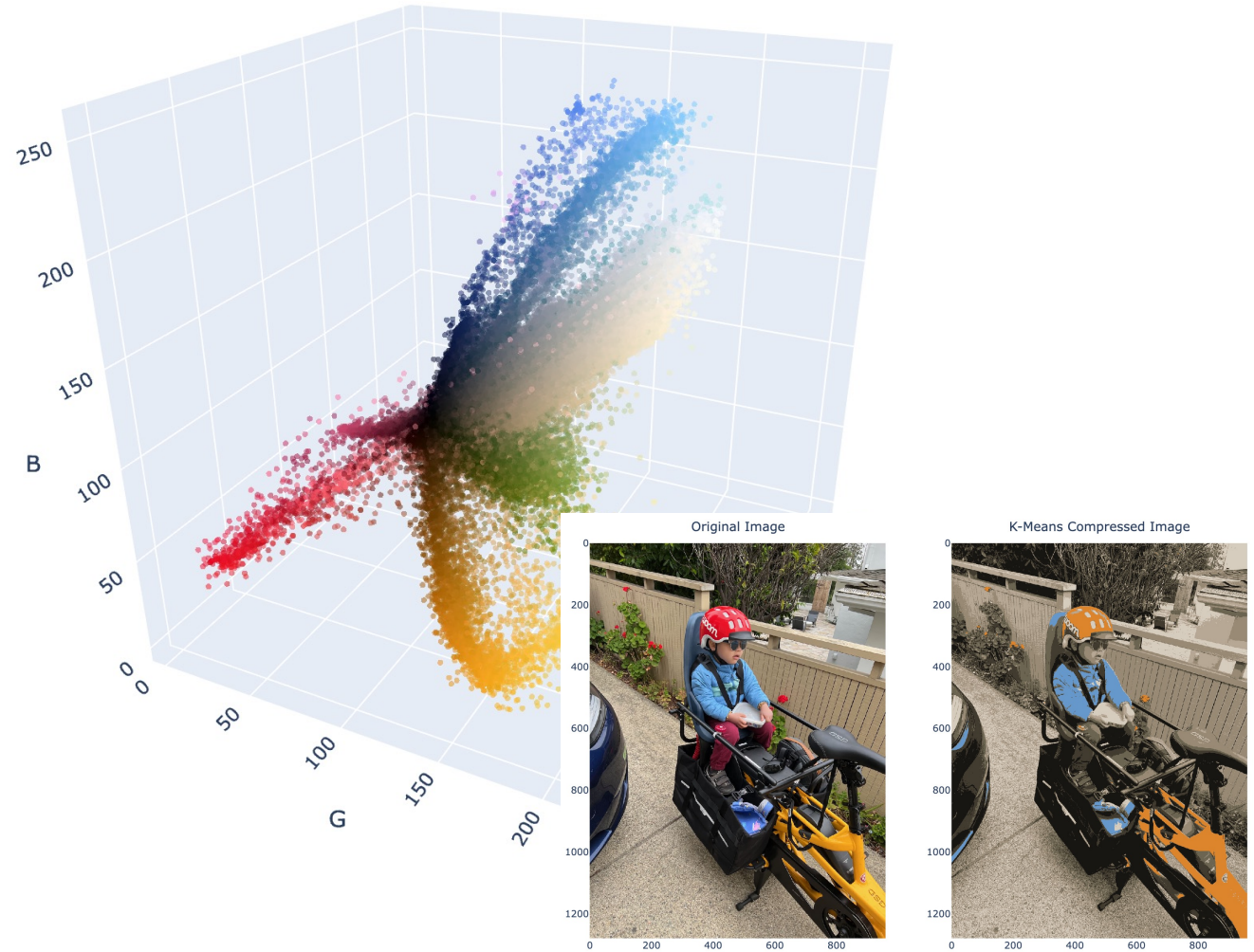
Pixels plotted in
RGB vector space.



Visualize Pixel Clustering

Demo

K-means on Pixels

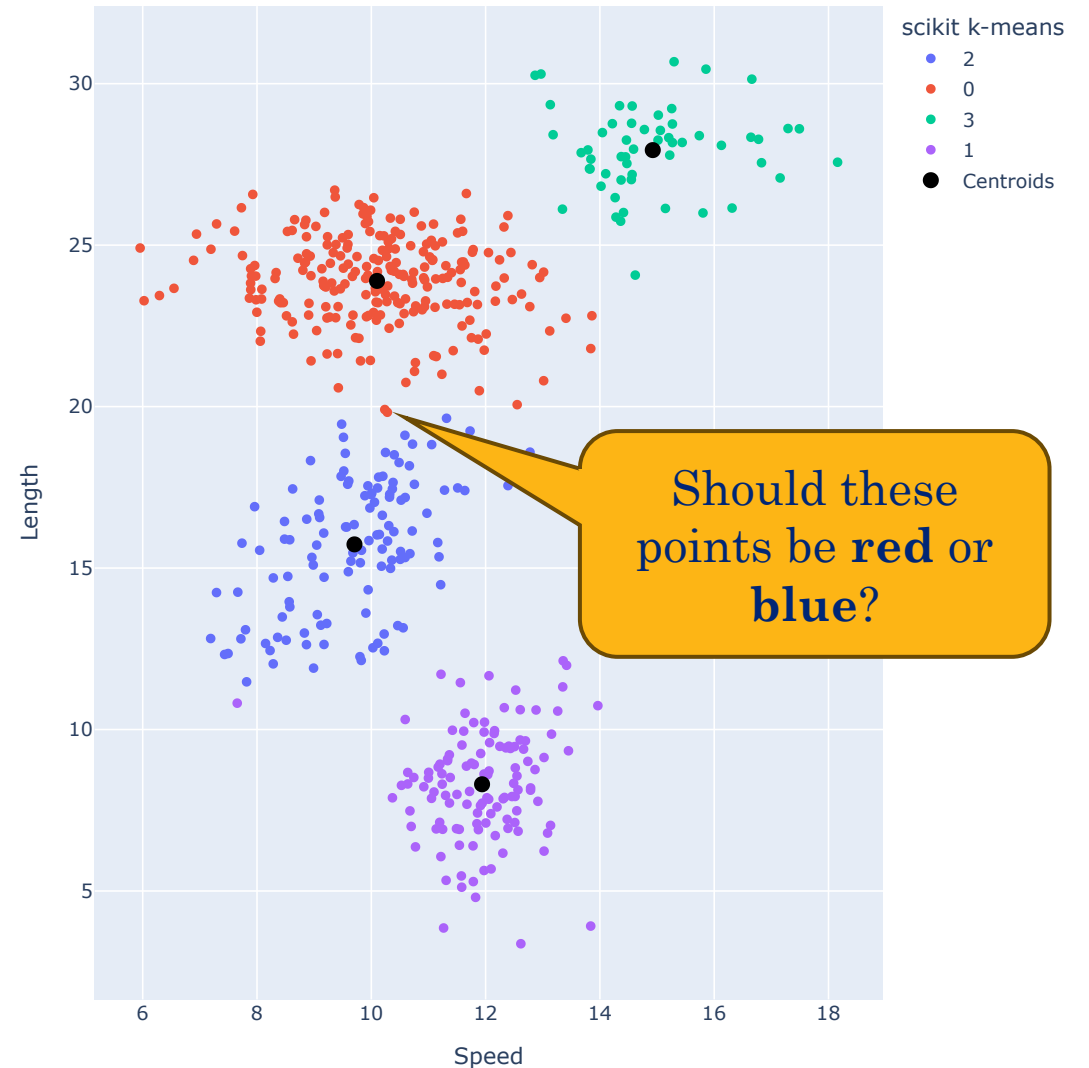


Hard Cluster Assignments

K-means assigns each data point to **exactly one cluster**.

Do all the points belong in exactly one cluster?

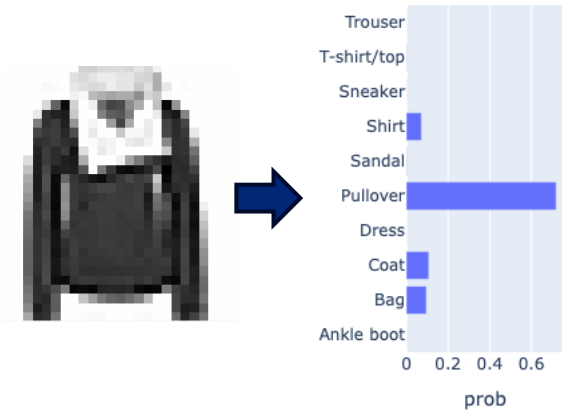
- **Maybe?**
- How could we at least measure the **uncertainty** in the **predictions**.



Uncertainty

ML is ultimately about **inference** – *making predictions*.

➤ Predictions are inherently *uncertain*.



Source of Uncertainty:

- **Epistemic Uncertainty (认识论不确定性)** (Reducible, 可约的/可消除的) – the *systematic uncertainty* that arises from a **finite training dataset** and our **modeling process**.
- **Aleatoric Uncertainty (偶然性不确定性)** (Irreducible, 不可约的/不可消除的) – the uncertainty that arises from observational *noise* in our **training data**.

Need a framework for uncertainty – *Probability!*

Probability (Review)

- K-means Clustering
 - Scikit-Learn
 - Lloyd's Algorithm
 - Pixel K-Means
 - Probability
 - Joint Distributions
 - Wake Word Example
- Questions



How do you interpret probabilities?

Probability

Probability provides a framework for quantifying and manipulating uncertainty.

The probability of an event is:

- **Frequentist View:** The long-run relative frequency of an event in identical repeated trials.
- **Bayesian View:** The degree of belief (or plausibility) assigned to the event given the available information.

Probability

假设我们要判断明天（3月8日）是否下雨（事件 A ）。

1. 频率学派的逻辑：计数法

频率学派关注的是事件发生的长期比例。它假设存在一个固定的、客观的概率 $P(A)$ 。

- 计算公式：

$$P(A) = \lim_{n \rightarrow \infty} \frac{n_A}{n}$$

其中， n 是过去 3 月 8 日的总天数， n_A 是其中下雨的天数。

- **做法：** 频率学派统计过去 100 年的 3 月 8 日，如果其中 20 天下了雨，那么频率学派会直接告诉你：“明天 3 月 8 日下雨的概率是 0.2”。
- **局限：** 它完全忽略了今天（3月7日）的云图、湿度等具体特征。

Probability

2. 贝叶斯学派的逻辑：更新法

贝叶斯学派不认为概率是固定的，它是通过**贝叶斯定理**来不断修正对事件的“信念”。

- 计算公式（贝叶斯定理）：

$$P(A|D) = \frac{P(D|A) \cdot P(A)}{P(D)}$$

- $P(A)$ ：先验概率 (Prior)，即基于历史记录（频率学派的结果），我们认为下雨的初始可能性。
- $P(D|A)$ ：似然 (Likelihood)，即在“下雨”的前提下，观测到今天这种气象特征（数据 D ）的可能性有多大。
- $P(A|D)$ ：后验概率 (Posterior)，即结合今天气象特征后，我们认为明天最终下雨的“修正后概率”。

Probability

做法

1. 先验

使用历史数据得到3月8日下雨先验概率 $P(A) = 0.2$ 。

2. 观测

今天（3月7日）气象云图显示乌云密布（数据 D ）。在历史上第二天（3月8日）下雨的日子里，这种云图出现的频率很高（似然 $P(D|A)$ 很大）。

3. 计算（更新修正）

通过公式，我们将初始信念 0.2 向上修正。比如，最终计算出的后验概率 $P(A|D)$ 可能变成了 0.8。

A brief review of the

Basics of Probability

The Joint Probability Distribution

Let X and Y be **discrete random variables** that take values:

$$X \in \{x_1, \dots, x_L\} \quad \text{and} \quad Y \in \{y_1, \dots, y_M\}$$

The joint probability function $p(X = x_i, Y = y_j)$ is the probability of the event $X = x_i$ and $Y = y_j$.

The joint probability satisfies the **following two properties**:

$$\forall x_i, y_j: p(X = x_i, Y = y_j) \geq 0 \quad \text{(Non-negativity)}$$

$$\sum_{i=1}^L \sum_{j=1}^M p(X = x_i, Y = y_j) = 1 \quad \text{(Normalization)}$$

$p(X = x_i, Y = y_j)$	y_1	y_2	
x_1	0.2	0.1	0.3
x_2	0.1	0.1	0.2
x_3	0.15	0.35	0.5
	0.45	0.55	1

The Joint Probability Distribution

Let X and Y be **discrete random variables** that take values

$$X \in \{x_1, \dots, x_L\} \quad \text{and} \quad Y \in \{y_1, \dots, y_M\}$$

The joint probability function $p(X = x_i, Y = y_j)$ is the probability of the event $X = x_i$ and $Y = y_j$.

The Sum Rule (Marginalization):
defines the distribution over a subset of the random variables.

$$\text{Marginal Probability } p(X = x_i) = \sum_{j=1}^M p(X = x_i, Y = y_j)$$

$p(X = x_i, Y = y_j)$	y_1	y_2	$p(X = x_i)$
x_1	0.2	0.1	0.3
x_2	0.1	0.1	0.2
x_3	0.15	0.35	0.5
$p(Y = y_i)$	0.45	0.55	1

Conditional Probability

We define the conditional probability as the chance of observing $Y = y_j$ given **we observed** $X = x_i$

$$p(Y = y_j | X = x_i) = \frac{p(X = x_i, Y = y_j)}{p(X = x_i)}$$

Example:

What is the $p(Y = y_1 | X = x_3)$?

$$\frac{\begin{array}{|c|c|c|} \hline x_3 & 0.15 & 0.35 \\ \hline \end{array}}{0.5} = \begin{array}{|c|c|} \hline 0.3 & 0.7 \\ \hline \end{array} \quad p(Y = y_i | X = x_3)$$

$p(X = x_i, Y = y_j)$	y_1	y_2	$p(X = x_i)$
x_1	0.2	0.1	0.3
x_2	0.1	0.1	0.2
x_3	0.15	0.35	0.5
$p(Y = y_i)$	0.45	0.55	1

Empirical Probability Distributions

Consider N trials $\{(X_1, Y_1), \dots, (X_N, Y_N)\}$ in which we sample (X, Y) .

Let n_{ij} be the number (count) of trials where $(X = x_i, Y = y_j)$, we can construct a discrete **empirical probability distribution**:

$$\hat{p}(X = x_i, Y = y_j) = \frac{n_{ij}}{N}$$

Frequentist View: In the limit as $N \rightarrow \infty$, the empirical probability distribution converges to the true probability distribution:

$$p(X = x_i, Y = y_j) = \lim_{N \rightarrow \infty} \hat{p}(X = x_i, Y = y_j)$$

The Product Rule of Probability

Recall the definition of the conditional:

$$p(Y = y_j \mid X = x_i) = \frac{p(X = x_i, Y = y_j)}{p(X = x_i)}$$

Multiply both sides by $p(X = x_i)$ you obtain the **product rule**:

$$p(X = x_i, Y = y_j) = p(Y = y_j \mid X = x_i) p(X = x_i)$$

The product rule is like a **chain rule** of probability – it allows us to **factorize** joint probability distributions into conditionals:

$$p(X, Y, Z) = p(Z \mid X, Y) p(X \mid Y) p(Y)$$

- *Notation*: we often drop x_i , y_j , and z_k for convenience.

Independent Random Variables

Two variables X and Y are independent if the joint probability factorizes:

$$X \perp Y \Rightarrow p(X, Y) = p(X) p(Y)$$

Using the definition of conditionals:

$$X \perp Y \Rightarrow p(Y|X) = \frac{p(Y, X)}{p(X)} = \frac{p(Y)p(X)}{p(X)} = p(Y)$$

Independent and Identically Distributed (IID) data (X_i, Y_i) is:

$$p(X_1, Y_1, \dots, X_N, Y_N) = \prod_{n=1}^N p(X_n, Y_n)$$

- We will often assume IID data (though it is often not true in practice).

Wake Word Example

- K-means Clustering
 - Scikit-Learn
 - Lloyd's Algorithm
 - Pixel K-Means
- Probability
 - Joint Distributions
- Wake Word Example

Questions

Example: Wake Words

A **wake word** is a verbal cue that triggers voice assistants to start actively listening.

Example: *“Alexa, set an alarm for 6:00AM”*



Most voice assistants **continuously** run a wake word detector model on every sound they hear.



Example: Wake Words

A **wake word** is a verbal cue that triggers voice assistants to start actively listening.

Example: *“Alexa, set an alarm for 6:00AM”*

Most voice assistants **continuously** run a wake word detector model on every sound they hear.



Example: Wake Words



How good does the wake word detector model need to be?

- To answer this question, we can model the **joint probability** of the **wake word detector** on an audio segment.

Before we listen to the audio segment, there are two random variables:

1. Was a wake word said: $X \in \{0, 1\}$ (0: no, 1: yes)
2. Was a wake word detected: $Y \in \{0, 1\}$ (0: negative, 1: positive)

Analyzing the Wake Word Detector

Let's assume wake words are rare occurring less than 0.01% of segments:

$$p(X = 1) = 0.0001 \quad \text{and} \quad p(X = 0) = 1 - p(X = 1) = 0.9999$$

Suppose the wake word model **detects 99% of the wake words**:

$$p(Y = 1 | X = 1) = 0.99$$

And **falsely detects (false positive)** a wake word just 0.1% of the time:

$$p(Y = 1 | X = 0) = 0.001$$

If the **detector is positive**, what is the chance that **it was a wake word**?

$$p(X = 1 | Y = 1) = \frac{p(Y = 1, X = 1)}{p(Y = 1)} = \frac{p(Y = 1 | X = 1) p(X = 1)}{p(Y = 1)}$$

Bayes' Theorem

Bayes' Theorem

Named after [Thomas Bayes](#) who was an 18th century statistician, philosopher, and minister.

$$p(A|B) = \frac{p(B|A)p(A)}{p(B)} = \frac{p(B|A)p(A)}{\sum_A p(B|A)p(A)}$$

- Derived from product rule: $p(A, B) = p(A|B)p(B) = p(B|A)p(A)$

Useful in Bayesian statistics:

- $p(A)$ is the **prior belief** about A
- $p(B|A)$ is the **likelihood** of an observation B given those beliefs
- $p(A|B)$ is our **posterior belief** about A given our observation B

假设你初始认为“硬币是公平的”的置信度 $p(A)$ 为 **0.999**（虽然不是 1，但已经非常接近绝对真理了）。这意味着你给“硬币作弊”留下了 0.001 的怀疑空间。

当发生“连续 10 次正面”这个现象 B 时：

- 公平假设 (A) 下：观测概率 $p(B|A) = 0.5^{10} \approx 0.000977$
- 作弊假设 ($\neg A$) 下：假设作弊者总是出正面，则观测概率 $p(B|\neg A) = 1$

我们将这些数字代入贝叶斯定理，计算观测后的信念 $p(A|B)$ ：

$$p(A|B) = \frac{p(B|A) \cdot p(A)}{p(B|A) \cdot p(A) + p(B|\neg A) \cdot p(\neg A)}$$

计算结果：

- 分子： $0.000977 \times 0.999 \approx 0.000976$
- 分母： $(0.000977 \times 0.999) + (1 \times 0.001) \approx 0.000976 + 0.001 = 0.001976$
- $p(A|B) \approx 0.000976/0.001976 \approx 0.494$

发生了什么？

发生了什么？

你看，仅仅因为这一个“连续 10 次正面”的证据，你对“硬币公平”的信念就从 **99.9%** 崩塌到了 **49.4%**！

这个结论非常有意思：

1. **信念的脆弱性：** 尽管你初始信念极强，但面对强大的数据证据（似然函数），你的信心在瞬间就能被“腰斩”。
2. **没有永远的绝对：** 只要你保留了 0.001 的怀疑空间，贝叶斯定理就像一个过滤器，它会无情地通过证据把你的信念修正到与现实匹配的位置。

这就是为什么统计学家常说：贝叶斯思维本质上是一种“极度理性”的怀疑论。

Analyzing the Wake Word Detector

We assumed rare wake words: $p(X = 1) = 0.0001$

A **true positive rate (recall)** of 99%: $p(Y = 1 | X = 1) = 0.99$

And a **false positive rate** of 0.1%: $p(Y = 1 | X = 0) = 0.001$

If the **detector is positive**, what is the chance that **it was a wake word**?

$$p(X = 1 | Y = 1) = \frac{p(Y = 1 | X = 1) p(X = 1)}{p(Y = 1 | X = 1) p(X = 1) + p(Y = 1 | X = 0) p(X = 0)}$$
$$p(X = 1 | Y = 1) = \frac{0.99 * 0.0001}{0.99 * 0.0001 + 0.001 * 0.9999} = 0.09 = 9\%$$

Is that good (enough)?

How would we improve it?



**How could we improve the
Wake Word Detector?**

Demo

Analysis of Recall and False Positive Rates

$P(\text{Wake} \mid \text{Detect})$ vs False Positive Rate for Perfect Recall



Bayesian Updates: Wake Word Detector

Prior: we assumed rare wake words $p(X = 1) = 0.0001$

After observing the test $Y = 1$ we **updated our belief** about the presence of the wake word using Bayes Rule:

$$p(X = 1 | Y = 1) = \frac{p(Y = 1, X = 1)}{p(Y = 1)} = \frac{p(Y = 1 | X = 1) p(X = 1)}{p(Y = 1)} = 0.09$$

Posterior: the updated belief $p(X = 1 | Y = 1)$ after the observation.

Bayesian Updating: the process of starting from a **prior belief** and then using **observations** to compute **posterior distribution** (updated belief) is central to **Bayesian machine learning**.

Lecture 3

K-Means and Probability

Reference Book Chapters:

- **Clustering:** Chapter 15.1 (k-means)
- **Probability:** Chapter 2.[1-2]